INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

MATERIA: Estructura de Datos

PROFESOR: Víctor Hugo Vázquez Herrera

CARRERA: Ingeniería en Sistemas Computacionales.

SEMESTRE: 3° Sabatino

GRUPO: "B"

Alumna: Fatima Itzel Martínez Salamanca

Matrícula: 257o03058

Práctica:Pilas Estáticas y Dinámicas en Java

Xalapa ver. 12 de Octubre del 2026

INSTITUTO TECNOLÓGICO SUPERIOR DE XALAPA

Introducción Teórica

Una Pila (Stack) es una estructura de datos lineal que opera bajo el principio LIFO (Last In, First Out / Último en entrar, primero en salir). El último elemento en ser insertado es el primero en ser extraído. Este comportamiento es fundamental en informática para la evaluación de expresiones aritméticas, llamadas a funciones y mecanismos de control.

• Pilas Estáticas: Se implementan mediante arreglos unidimensionales de tamaño fijo. Requieren control explícito de desbordamiento (Overflow) con isFull() y subdesbordamiento (Underflow) con isEmpty().
• Pilas Dinámicas: Se implementan mediante nodos enlazados mediante referencias. Su tamaño es dinámico y depende de la memoria RAM disponible, por lo que no requieren un límite estático ni isFull().

Representación Gráfica del Comportamiento LIFO

Esquema gráfico del funcionamiento de las operaciones Push, Pop y Peek en las pilas:

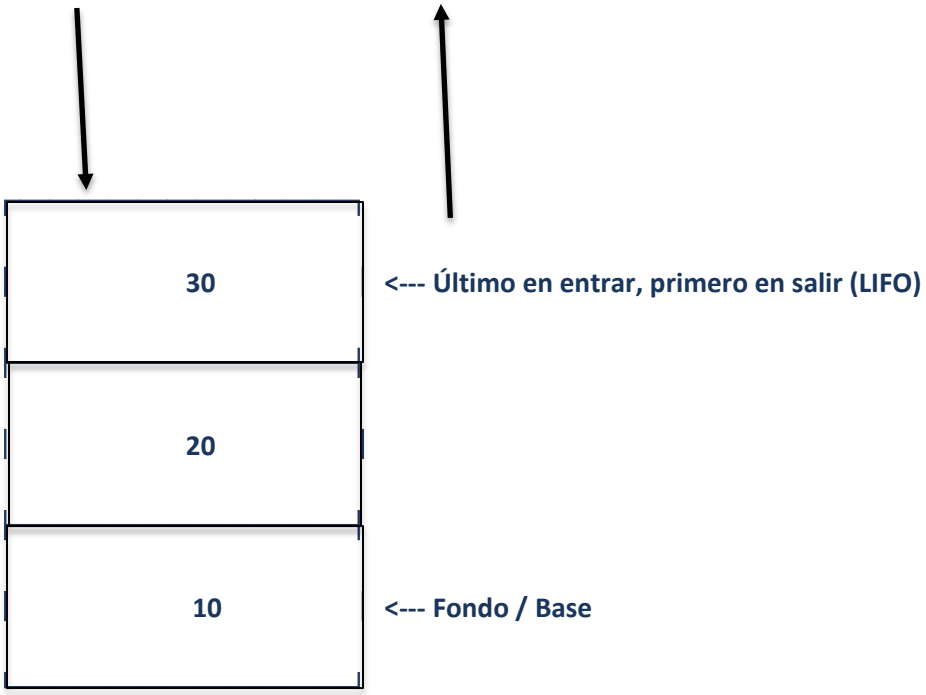
Operación Push	Estado LIFO (Tope -> Base)	Operación Pop

push(10), push(20), push(30)	`[ 30 ] (Tope)`
	`[ 20 ]`
	`[ 10 ] (Base)`
		pop() extrae el 30

push(40), push(50)	`[ 50 ] (Tope)`
	`[ 40 ]`
	`[ 30 ] ...`
		pop() extrae el 50

peek()	`Consulta el tope sin modificar la pila`	isEmpty() / isFull()

ENTRADA /SALIDA (Tope / Cima)



Código Fuente Implementado en Java

Pila Estática (PilaSimple.java)

```
package Estructuras_lineales;

public class PilaSimple {
    private int[] datos;
    private int tope;

    public PilaSimple() {
        datos = new int[5];
        tope = -1;
    }

    public void push(int x) {
        if (isFull()) {
            System.out.println("Desbordamiento: La pila estatica esta llena.");
            return;
        }
        datos[++tope] = x;
    }

    public void pop() {
        if (isEmpty()) {
            System.out.println("Subdesbordamiento: La pila esta vacia.");
```

```
            return;
        }
        System.out.println("Elemento extraido: " + datos[tope--]);
    }

    public void mostrar() {
        if (isEmpty()) {
            System.out.println("Pila vacia.");
            return;
        }
        System.out.print("Pila estatica (Tope -> Base): ");
        for (int i = tope; i >= 0; i--) {
            System.out.print(datos[i] + " ");
        }
        System.out.println();
    }

    public boolean isEmpty() {
        return tope == -1;
    }

    public boolean isFull() {
        return tope == datos.length - 1;
    }

    public void peek() {
        if (isEmpty()) {
            System.out.println("Pila vacia: no hay tope.");
        } else {
            System.out.println("Dato en el tope: " + datos[tope]);
        }
    }
}
```

Lanzador Pila Estática (Lanzador_pilasimple.java)

```
package Estructuras_lineales;

public class Lanzador_pilasimple {
    public static void main(String[] args) {
        PilaSimple p = new PilaSimple();
        System.out.println("Pila estatica");
        System.out.println("Esta vacia: " + p.isEmpty());
        p.push(10);
        p.push(20);
        p.push(30);
        p.mostrar();
        p.peek();
        p.pop();
        p.mostrar();
        System.out.println("\n Llenar la pila");
```

```
        p.push(30);
        p.push(40);
        p.push(50);
        System.out.println("Esta llena: " + p.isFull());
        p.push(60);
        p.mostrar();
        p.peek();
        System.out.println("\n Vaciar pila");
        while (!p.isEmpty()) {
            p.pop();
        }
        System.out.println("Esta vacia: " + p.isEmpty());
        p.peek();
        p.pop();
    }
}
```

Nodo (Nodo.java)

```
package Estructuras_lineales;

public class Nodo {
    int dato;
    public Nodo siguiente;

    public Nodo(int x) {
        this.dato = x;
        this.siguiente = null;
    }
}
```

Pila Dinámica (PilaDinamica.java)

```
package Estructuras_lineales;

public class PilaDinamica {
    private Nodo cima;

    public PilaDinamica() {
        cima = null;
    }

    public void push(int x) {
        Nodo nuevo = new Nodo(x);
        nuevo.siguiente = cima;
        cima = nuevo;
    }

    public void pop() {
        if (!isEmpty()) {
            System.out.println("Sacaste de la pila: " + cima.dato);
```

```
            cima = cima.siguiente;
        } else {
            System.out.println("Pila dinamica vacia: no hay elementos para sacar.");
        }
    }

    public void mostrar() {
        Nodo actual = cima;
        System.out.print("Pila dinamica (Cima -> Fondo): ");
        while (actual != null) {
            System.out.print(actual.dato + " ");
            actual = actual.siguiente;
        }
        System.out.println();
    }

    public boolean isEmpty() {
        return cima == null;
    }

    public void peek() {
        if (isEmpty()) {
            System.out.println("Pila vacia: no hay un dato para consultar.");
        } else {
            System.out.println("Dato en la cima: " + cima.dato);
        }
    }
    // Nota: La pila dinámica NO utiliza isFull() ya que su capacidad depende de la
RAM.
}
```

Lanzador Pila Dinámica (Main.java)

```
package Estructuras_lineales;

public class Main {
    public static void main(String[] args) {
        PilaDinamica p = new PilaDinamica();
        System.out.println("Pila Dinamica");
        System.out.println("Esta vacia: " + p.isEmpty());
        p.push(10);
        p.push(20);
        p.push(30);
        p.mostrar();
        p.peek();
        p.pop();
        p.mostrar();
        System.out.println("\n Agregar mas Nodos");
        p.push(30);
        p.push(40);
        p.push(50);
```

```
            p.push(60);
            p.mostrar();
            p.peek();
            System.out.println("\n Vaciar pila");
            while (!p.isEmpty()) {
                p.pop();
            }
            System.out.println("Esta vacia: " + p.isEmpty());
            p.peek();
            p.pop();
        }
    }
```

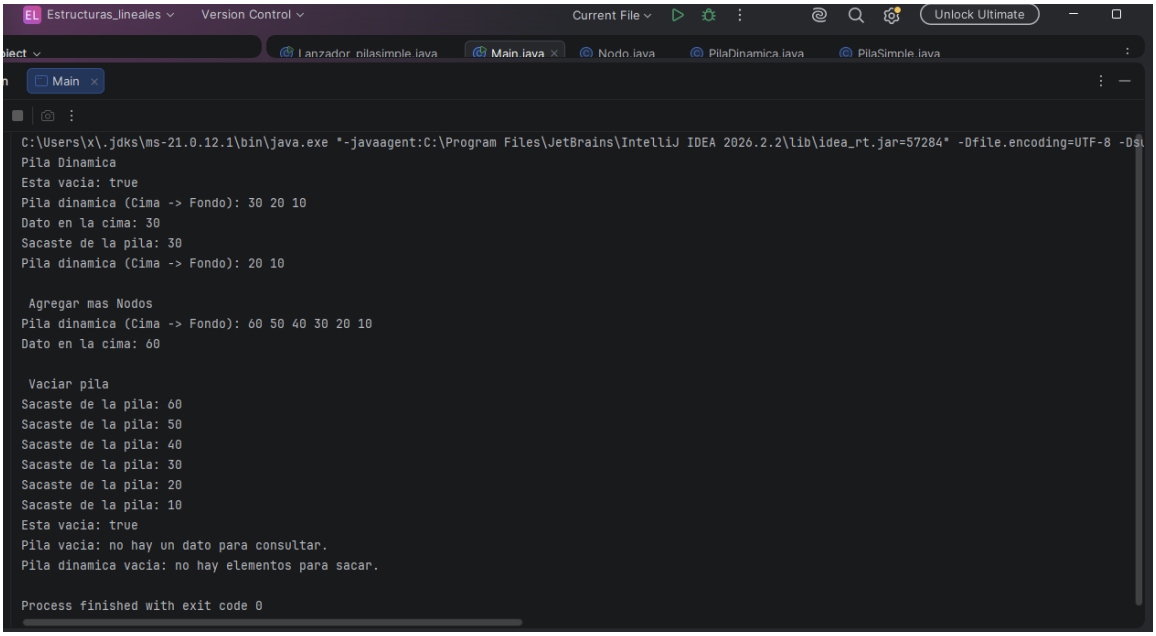
Pruebas de Funcionamiento y Evidencias de Ejecución

A continuación se insertan las capturas de pantalla de la ejecución de las pruebas en el entorno de desarrollo IntelliJ IDEA:

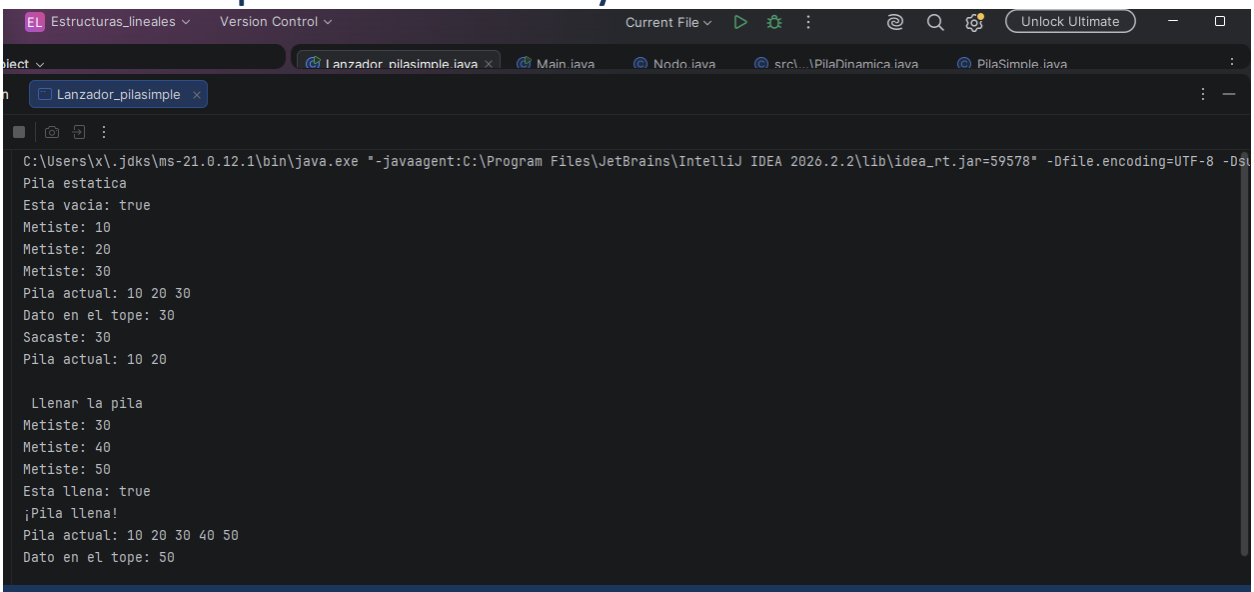
4.1 Evidencia de Ejecución - Pila Dinámica (Main.java)

4.2 Evidencia de Ejecución - Pila Estática (Lanzador_pilasimple.java)

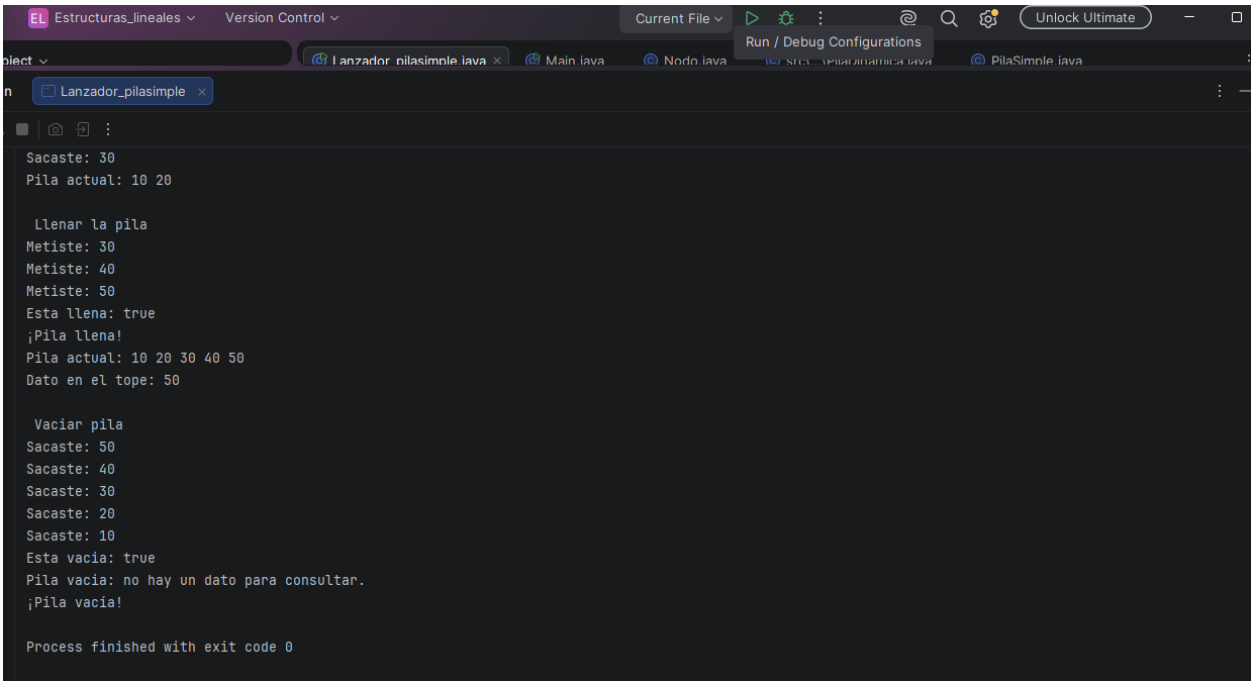
Captura de consola correspondiente a la Pila Estática:



5. Análisis Comparativo: Rendimiento y Uso de Memoria



Criterio	Pila Estática	Pila Dinámica
Memoria	Arreglo contiguo de tamaño estático.	Nodos enlazados dispersos en el Heap.



Capacidad	Limitada y fija (requiere validación isFull).	Flexible, limitada solo por la RAM (sin isFull).
Eficiencia	Acceso directo por índice O(1).	Navegación secuencial por referencias O(1) en extremos.
Control de Errores	Manejo de Overflow y Underflow.	Manejo de Underflow (pila vacía).

6. Conclusiones Personales

1. Hacer esta práctica nos ayudó bastante a ver cómo funcionan las pilas por dentro y por qué cambia tanto usar un arreglo fijo frente a trabajar con nodos y apuntadores en Java.

2. Aprendí que validar siempre cosas como `isEmpty`, `isFull` o checar el `peek` nos ahorra dolores de cabeza y evita que el programa truene con excepciones cuando la pila está vacía o se pasa de capacidad.

3. Las pruebas con los lanzadores nos sirvieron para comprobar que el orden LIFO se cumple tal cual, metiendo y sacando datos sin perder el control de la cima o del tope.